

An Ant Colony System for Passenger Assignment and Route Design of Urban Customized Bus With Flexible Pick-Ups and Drop-Offs

Wen-Jin Qiu^{id}, *Student Member, IEEE*, Zhan-Xian Liang^{id}, *Student Member, IEEE*,
Xiao-Min Hu, *Senior Member, IEEE*, Jun Zhang^{id}, *Fellow, IEEE*,
and Wei-Neng Chen^{id}, *Senior Member, IEEE*

Abstract—Urban customized bus (UCB) has become an important mode of urban transportation in recent years. To further enhance the applicability of UCB, we develop a UCB model that accommodates flexible pick-ups and drop-offs in this article. In our model, when a bus passes through a station, it can drop off passengers whose destination is exactly this station and pick up new passengers as long as constraints are satisfied. By eliminating fixed boarding and alighting regions that were commonly adopted in prior studies, we can provide travel services for both prebooked and real-time orders across broader temporal and spatial ranges. Under such a scenario, it is more challenging to decide when and where to pick up which passenger with which bus, and which route the bus chooses. To solve these intractable problems, we proposed an approach including passenger assignment and route design. For passenger assignment, we devise an ant colony system (ACS) method and a greedy method to assign prebooked and real-time passengers to buses, respectively. ACS represents the correlation among different passengers numerically through a pheromone matrix and updates it iteratively based on the good results. Therefore, prebooked passengers with higher correlations are more likely to be assigned to the same buses. The greedy method always selects passengers with the highest heuristic value to have a fast response in real-time passenger assignment. For route design, we adapt a Dijkstra-based greedy method to efficiently design routes for each bus in both static and dynamic situations. At last, experiments based on both generated data and real data demonstrate the superiority of the proposed method.

Index Terms—Ant colony optimization (ACO), customized bus, passenger assignment, route design.

I. INTRODUCTION

URBAN customized bus (UCB) is an emerging and important mode of public transportation that provides passengers with personalized and flexible demand–response public transport service [1], [2]. It takes advantage of the mobile Internet, collecting precise travel demands of passengers through online platforms [3]. Based on the obtained data, UCB operators can abandon the fixed routes and timetables typical of traditional buses. Instead, they can arrange buses to follow specific routes at suitable times, catering to passengers with similar origin–destination (OD) pairs and departure schedules [4]. UCB is a promising mode of public transportation. For operators, the operational mode of UCB enables them to optimize bus deployment and routes with precise demand data, thereby enhancing operational efficiency. For passengers, UCB offers a convenient, punctual, and comfortable travel experience at a relatively low cost [5]. Meanwhile, from the view of city managers, UCB can incentivize more citizens to opt for public transportation, thus improving urban road network efficiency and reducing traffic congestion [6].

Recently, research on UCB has become a trending topic [7], [8], [9] and can be roughly classified into two categories. The first category is travel demand extraction of UCB. To extract the potential travel demands of UCB, data mining techniques such as property analysis and clustering are utilized to estimate potential UCB travel demands based on historical travel data from other modes of public transportation or private vehicles [7], [10], [11]. The second one is the optimization of bus operations, including stop planning, passenger assignment, route design, timetable deployment, and so on [2], [8], [12], and [13]. Methods such as mixed-integer programming, heuristic, meta-heuristic, and machine learning are employed to optimize these procedures either individually or collaboratively.

However, in most existing studies, UCB operates based on a simplified model where buses can only pick up passengers in fixed boarding regions while drop them off in fixed alighting regions [3], [4], [5], [8], [11], [13], [14], [15]. This model is tailored for UCB operating during commuting

Received 26 July 2025; accepted 16 December 2025. Date of publication 31 December 2025; date of current version 26 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62376097; in part by Guangzhou Science and Technology Project under Grant SL2023A04J00122; and in part by the National Research Foundation of Korea (NRF) grant funded by the Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2025-00555463, Grant RS-2025-25456394, and Grant RS-2025-25460542. This article was recommended by Associate Editor R. Mallipeddi. (Corresponding authors: Xiao-Min Hu; Wei-Neng Chen.)

Wen-Jin Qiu and Wei-Neng Chen are with the School of Computer Science and Engineering and the State Key Laboratory of Subtropical Building and Urban Science, South China University of Technology, Guangzhou 510006, China (e-mail: ufeqnm18@foxmail.com; cschenwn@scut.edu.cn).

Zhan-Xian Liang is with ByteDance Company Ltd., Beijing 100098, China (e-mail: 1359672919@qq.com).

Xiao-Min Hu is with the Department of Computer Science, Guangdong University of Technology, Guangzhou 510006, China (e-mail: xmhu@ieee.org).

Jun Zhang is with Nankai University, Tianjin 321004, China, and also with Hanyang University, ERICA, Ansan 15588, South Korea (e-mail: junzhang@ieee.org).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TSMC.2025.3648151>.

Digital Object Identifier 10.1109/TSMC.2025.3648151

hours, particularly in cities with distinguishable homeplaces and workplaces. Since these two types of regions are usually separate and situated far apart from each other, the passenger assignment and route design processes for commuting UCB are straightforward to address. Buses pick up as many passengers as possible at stops within the boarding regions. Then, buses travel directly to the alighting regions without detailed consideration of the intermediate routes. Upon reaching the alighting regions, buses drop off passengers at their respective destination stops.

Commuting UCB is easy for modeling and optimizing, but its limitations are also obvious. Regarding time, commuting hours comprise only a fraction of the working day. Outside of these hours, the OD pairs of passengers are not as concentrated as during commuting times. In this situation, commuting UCB is not suitable for serving passengers whose OD pairs are scattered across the entire city. In terms of spatial considerations, distinguishing between residential and work regions in cities can be challenging. For instance, in many Chinese cities undergoing a blend of old city revitalization and new city expansion, this distinction is blurred. In such cases, commuting UCB, with separate boarding and alighting locations, may not adequately meet citizens' travel needs even during commuting hours. Moreover, some passengers need to travel within boarding regions and alighting regions even during commuting times. Considering these diverse travel demands can enhance UCB utilization and increase revenue. Due to these limitations, there is an urgent need for a more flexible UCB model that can be adapted to different scenarios.

To bridge these gaps, we develop a more comprehensive, flexible, and practical model for the operation of UCB in this study. Our model enables buses to provide flexible pick-up and drop-off services during their operations. It means that when a bus passes through a station, it can not only drop off passengers whose destination is that station but also pick up new passengers without violating any constraints. With this flexible operational mode, UCB can be utilized on more occasions, including typical peak commuting hours, off-peak periods, holidays, and across a wider range of locations without the need to distinguish between boarding and alighting areas. Moreover, it can also be applied to dynamic traffic flow situations to respond to real-time ride requests.

To provide flexible pick-up and drop-off service, the intractable passenger assignment and route design problems need to be solved. For passenger assignment, we need to assign a set of appropriate passengers to each bus. We employ an ant colony system (ACS) method [16] and a greedy method to handle passenger assignment for prebooked and real-time passengers, respectively. In ACS, we devise an encoding scheme that represents the sequences in which prebooked passengers are assigned to buses. With this proposed encoding scheme, the correlation among different passengers can be presented numerically with the pheromone and heuristic information. Subsequently, ACS is utilized to select appropriate passengers for bus pick-up, leveraging both pheromone and heuristic information. In the greedy method, the heuristic values of real-time passengers who meet all the constraints are calculated. Then, passengers with the highest heuristic values are selected to improve the efficiency of real-time response. For

the route design of buses, a Dijkstra-based [17] greedy method is proposed to design routes for buses. When designing routes, the proposed method always selects the shortest routes found by the Dijkstra algorithm. Since route design is frequently needed, a greedy method can ensure the solution's efficiency. Moreover, the proposed method can also lead to satisfactory results. That is because passengers assigned to the same bus usually have similar itineraries, shortest paths found by the Dijkstra algorithm are usually good results.

The contributions of this article are listed as follows.

- 1) To expand the usage of UCB, we develop a more flexible UCB model in which buses can pick up and drop off passengers at every station along their routes for both prebooked and real-time passengers, considering both the operational cost and the passenger experience.
- 2) We solve the passenger assignment problem for prebooked and real-time passengers by employing ACS and a greedy method, respectively. A Dijkstra-based greedy routing method is utilized to design routes during passenger assignment.
- 3) We conduct experiments using both generated data and real data collected by a bus operating company. The results obtained from these two types of data consistently demonstrate the superiority of the proposed method.

The rest of this article is organized as follows. Section II briefly introduces some background about UCB, as well as ACO. Section III illustrates the proposed UCB model with flexible pick-ups and drops-off. Section IV provides a detailed description of the proposed passenger assignment method and the Dijkstra-based route design method. Section V presents some experiments on generated data and real data to verify the effectiveness of the proposed method. The conclusion is presented in Section VI.

II. BACKGROUND

A. Urban Customized Bus

The last decade has witnessed the emergence of UCB. Because of its convenience, comfort, punctuality, and economy, UCB has become an important mode of public transportation [1], [2]. Currently, studies related to UCB can be roughly classified into two categories: travel demand extraction [7], [10], [18] and bus operation optimization [8], [12], [13].

In travel demand extraction, researchers mined the travel demands of UCB based on historical travel data from other modes of public transportation or private vehicles. Guo et al. [10] proposed a methodology to explore the potential travel demand of UCB according to the bus smart card data (SCD) in Beijing. Trip chain generation was used to preprocess raw data and extract important information about the passengers at first. Then, OD recognition is used to determine the station name and geographical location of the origin and destination. Lastly, travel mode comparison is proposed to analyze the potential demand of UCB. Liu et al. [18] presented a visual analytical approach to schedule customized shuttle buses based on 93 050 employees' car-hailing reimbursement records. The trajectories of these records are visualized, providing an intuitive guide to humans. Clustering methods are

then used to determine the travel directions and initialize the candidate shuttle bus stations. At last, the information about the candidate shuttle bus stations and routes conveyed by the map view is refined based on the expertise of domain experts.

The operational procedures of buses that need optimization include, but are not limited to, passenger assignment, stop planning, route design, and timetable deployment. Chen et al. [12] developed an integrated optimization method for the deployment of customized bus stops, route design, and timetable development. They formulated an objective function that considers passengers' inconvenience, the total number of customized bus stops across all routes, and the total routing cost to evaluate the overall cost of customized bus service. There are also some researchers tried to integrate demand extraction into the optimization of bus operations, planning stops, and designing routes for UCB based on travel demands extracted from electronic registration identification (ERI) or smart cards data [5]. Xia et al. [11] optimized the operation of UCB based on travel demands extracted from ERI data of private cars. They first extracted private car commuting trips from ERI data based on the properties of temporal similarity, spatial similarity, and high frequency. Then, the obtained travel demands were used as the travel demands for designing customized bus schemes.

Although there are numerous studies related to UCB, most of them are based on a commuting model. Only a few studies using models that get rid of fixed boarding and alighting regions [6], [19]. In [6], a UCB model that considers transfer, holding control, and a heterogeneous fleet was proposed. Passengers can be dropped off and picked up in intermediate transfer stations. An improved particle swarm optimization algorithm was used to solve the model, considering operational cost and passenger experience. In [19], a static on-demand bus routing problem was introduced, taking bus station assignment into account. Each bus can pick up and drop off passengers on its route according to the demand of passengers. A large neighborhood search was used to minimize the total user ride time of the bus routing problem. In our model, buses provide an origin-to-destination service for both prebooked and real-time passengers without transfers, enhancing convenience for passengers. Additionally, we factor in the boarding and alighting times of passengers to make the model more practical. In terms of objectives, we account for both operational cost and passenger experience, similar to [6]. However, in terms of passenger experience, we adopt a more comprehensive approach by considering more factors.

B. Ant Colony Optimization

ACO stands as one of the most prominent metaheuristic algorithms [20], [21], with successful applications across numerous real-world problems [22], [23], including UCB operations [4], [24]. Originally developed by Colnari et al. [25], ACO draws inspiration from the foraging behavior of ant colonies. ACO combines stochastic processes and heuristic guidance to optimize solutions iteratively. When solving a specific problem, an artificial ant generates a candidate solution by continuously selecting nodes in the modeled graph and

constructing a complete path. During the process, an artificial ant chooses nodes in a biased random way, guided by the amount of pheromone information and heuristic information in the graph. The result of the selection process also has a counter effect on the density of pheromone information in the graph because we need to update the pheromone according to the quality of the constructed solutions. As the iteration progresses, the density of pheromones in the graph and the constructed complete paths gradually converge, resulting in optimized solutions to the original problem. In this article, we apply ACS, recognized as one of the most effective ACO implementations, to optimize the passenger assignment process within our proposed UCB model.

III. UCB WITH FLEXIBLE PICK-UPS AND DROP-OFFS

In this section, we provide a detailed description of the proposed UCB model with flexible pick-ups and drop-offs. First, we introduce the three fundamental components of the proposed model: road network, buses, and passengers. Next, we present the relevant assumptions and constraints of the proposed model. Finally, we provide the solution presentation for the model and the objective function. All notations used in this article are listed in Table SI in the supplementary material.

A. Basic Elements

During the operation of UCB, the road network, buses, and passengers are the three fundamental components that must be considered. In the following description, we present each of these three components of the proposed model one by one.

1) *Road Network*: The road network is the operational environment of UCB. In this article, an undirected connected graph $G = (V, E)$ is used to describe the road network with multiple stations. Let V represent the set of stations, and let E contain all the connections between adjacent stations. The station $v_{\text{dep}} \in V$ serves as the bus depot, from which buses initiate their routes and ultimately return. An edge $e_{i,j} \in E$, connecting bus station v_i and v_j , has an associated weight $T(v_i, v_j)$, representing the average travel time on $e_{i,j}$, which is used for scheduling. However, when buses traverse $e_{i,j}$, the actual travel time is calculated by multiplying $T(v_i, v_j)$ by a sampled value drawn from a truncated normal distribution with a mean of 1 and a standard deviation of std.

2) *Buses*: Buses are responsible for picking up passengers from their origin and transporting them to their destinations through the edges of the road network. Unlike conventional buses [26] and UCB in [6], where transfers are considered, buses in our model offer origin-to-destination services. Additionally, buses in our model also provide flexible pick-up and drop-off services. It means that when a bus passes through a station, it can not only drop off passengers whose destination is that station but also pick up new passengers as long as constraints are not violated. For a bus $b_k \in B$, relevant variables can be represented by a tuple as shown in the following equation:

$$b_k = (\text{Cap}_k, \text{Rou}_k) \quad (1)$$

where Cap_k is the number of passengers assigned to b_k and Rou_k is the route of b_k designed based on travel demands of

passengers. For better differentiation, we use route to refer to bus-related routes and path to refer to passenger-related routes in this article.

3) *Passengers*: Passengers submit their travel demands in advance or in real-time and expect to receive satisfactory travel services. For a passenger $p_i \in P$, the relevant variables can be represented as an eight-tuple shown in the following equation:

$$p_i = (\text{ori}_i, \text{dest}_i, tw_i^s, tw_i^e, \text{spath}_i, t_i^{\text{pick}}, t_i^{\text{drop}}, \text{path}_i). \quad (2)$$

The first four variables, ori_i , dest_i , tw_i^s , and tw_i^e , correspond to the origin, destination, start time, and end time of the boarding time window of p_i , respectively. With ori_i , dest_i , and the road network, we can get the shortest path spath_i of p_i . The following two variables, t_i^{pick} and t_i^{drop} , are used to record the time when the passenger is picked up and dropped off by buses, respectively. The last variable, path_i , is used to record the actual travel path of p_i . In our model, we only consider the boarding time window since passengers prioritize reaching their destinations as quickly as possible. Therefore, calculating the detour ratio is more suitable than generating an alighting time window. For prebooked passengers who submit their travel demands in advance, we have full knowledge of all their demand details and assign them beforehand using ACS. For real-time passengers, we only obtain their demand information after they submit their travel requests at tw_i^s .

B. Assumptions and Constraints

1) *Assumptions*: Our model makes the following assumptions.

- 1) Buses can visit multiple stations to pick up and drop off passengers, but each bus can visit each station once to prevent extra detours that could result in additional cost. Buses initiate their routes and ultimately return to bus depots.
- 2) Only the time traveled along the edges is considered when calculating $T(\text{Rou}_k)$. According to [27], buses consume much less fuel when stationary compared to when in motion. Therefore, when buses stop at stations for picking up and dropping off passengers, the fuel consumed is not considered.
- 3) The travel time between any pair of adjacent stations, $T(v_i, v_j)$, follows a truncated normal distribution. A normal distribution was adopted in [28] to model path travel times in urban areas and was validated using data collected in Wuhan, China. In this study, we impose a lower bound (0.7) and an upper bound (1.8) to the normal distribution to constrain the range of possible travel times.
- 4) Buses are equipped with two doors for simultaneous boarding and alighting. Nowadays, even smaller buses, such as the BYD K5 electric bus, which accommodates only 10–17 seats, are designed with two doors. For relevant time, we refer to [14], allocating 10 s for bus stopping (t_s) and restarting (t_r), and 2.6 s for each passenger boarding and alighting (t_b and t_a).

2) *Constraints*: The first constraint pertains to the capacity limitation for buses. We cannot assign passengers to a bus

when it is full. It can be represented as (3), where the maximum capacity of buses is denoted as $\text{Cap}_{\max}$

$$\text{Cap}_k \leq \text{Cap}_{\max}, \quad \text{where } b_k \in B. \quad (3)$$

The second constraint is the maximum travel time limitation for buses. It can be formulated as (4), where $T_{\max}$ denotes the maximum travel time of buses

$$T(\text{Rou}_k) \leq T_{\max}, \quad \text{where } b_k \in B. \quad (4)$$

The third constraint ensures passengers reach their destinations without excessive detours. We define the detour ratio for a passenger $p_i \in P$ as the difference between t_i^{drop} and t_i^{pick} , divided by the shortest-path travel time $T(\text{spath}_i)$. As shown in (5), an upper bound $\text{Dtr}_{\max}$ is introduced to prevent excessively detour when serving p_i

$$\frac{t_i^{\text{drop}} - t_i^{\text{pick}}}{T(\text{spath}_i)} \leq \text{Dtr}_{\max}, \quad \text{where } p_i \in P. \quad (5)$$

The fourth constraint aims to minimize the number of reached intermediate stops. Let $|\text{path}_i|$ denote the number of stations visited by passenger p_i along path_i and $|\text{spath}_i|$ represent the number of stations along spath_i . The station ratio is defined in (6), where $\text{Sta}_{\max}$ denotes the maximum allowable station ratio

$$\frac{|\text{path}_i|}{|\text{spath}_i|} \leq \text{Sta}_{\max}, \quad \text{where } p_i \in P. \quad (6)$$

C. Solution Presentation and Objective Function

To find a solution to the proposed model, we need to serve all passengers and decide when and where to pick up which passenger with which bus and which route the bus chooses. In this case, the road network, buses, and passengers couple with each other. To better handle the relationship among these three basic elements, we define a combined order to record relevant decision variables that are executed by a bus $b_k \in B$. Specifically, a combined order can be represented as a tuple that is expressed in (7). The first decision variable Rou_k is the route of b_k , corresponding to the second variable in (1). The two following variables PList_k and RList_k represent the passenger assignment sequences for bus b_k , recording prebooked and real-time passengers, respectively. For a more intuitive understanding, Fig. 1 is provided below to visualize a combined order. In Fig. 1, Rou_k begins and ends at the bus depot v_{dep} (where passengers can also be picked up and dropped off). Bus b_k traverses Rou_k to pick up passengers recorded in PList_k . During this process, when b_k arrives at a station, passengers whose destination is this station will be dropped off, while real-time passengers originating at this station may be picked up (if constraints are satisfied) and added to RList_k . After picking up real-time passengers, Rou_k may be modified accordingly to serve them

$$\text{CO}_k = (\text{Rou}_k, \text{PList}_k, \text{RList}_k). \quad (7)$$

In our work, the solution of the proposed model can be represented by a set of combined orders, as shown in (8). The exact number of the combined orders, that is, the number

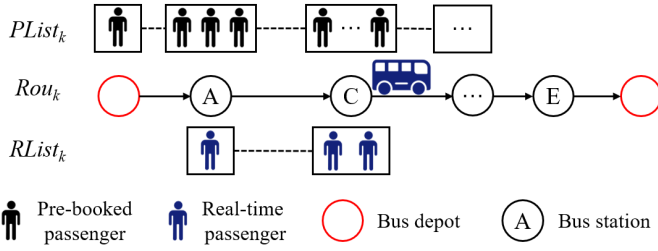


Fig. 1. Visual example of a combined order.

of buses needed to server all passengers, is determined by the optimization process of passenger assignment and route design, which will be described in detail in Section IV

$$S = (CO_1, CO_2, \dots, CO_k, \dots). \quad (8)$$

The objective function used to evaluate the obtained solution comprises two components: the cost of buses and the cost of passengers. The operating cost of buses consists of three components: fixed cost, variable cost, and penalty cost. The fixed cost covers expenses such as employee salaries. It is not related to the travel time of buses. In contrast, variable costs are directly related to travel time. The primary component is the fuel cost, which rises in proportion to the travel time. Given that the fuel consumption of the bus is minimal while it waits for passengers at stations, we only factor in the travel time on the edges of the road network to calculate the variable cost, aligning with assumption 2. The penalty cost is incurred when the running time of a bus exceeds $T_{\max}$. This situation is rare, since buses are required to return to the depot from the last station under worst case traffic conditions (the upper bound value of the truncated normal distribution) when scheduling. However, it remains possible due to the inherent travel time uncertainties throughout the journey. Therefore, the total operational cost of a bus is represented by (9), where C_1 , C_2 , and C_3 are three coefficients used to calculate the fixed cost, variable cost, as well as the time-out penalty

$$\text{Cost}_B = \sum_{k=1}^{|S|} (C_1 + C_2 \times T(\text{Rou}_k) + C_3 \times \text{TO}_k) \quad (9)$$

$$\text{where } \text{TO}_k = \begin{cases} 1, & \text{if } T(\text{Rou}_k) > T_{\max} \\ 0, & \text{otherwise.} \end{cases}$$

The cost associated with passenger experience is defined by (10). In this formulation, the variables x_i , y_i , and z_i account for three dimensions of passenger experience, and C_4 , C_5 , and C_6 are corresponding penalty coefficients. The first variable x_i ensures passengers are picked up within their designated boarding time window. The second variable y_i maintains the detour ratio within acceptable limits. The third variable z_i limits the number of intermediate stations passengers encounter. The upper limits $\text{Dtr}'_{\max}$ and $\text{Sta}'_{\max}$ are set to values lower than $\text{Dtr}_{\max}$ and $\text{Sta}_{\max}$ to better elevate the overall passenger experience. Notably, for unserved real-time passengers, all three variables are set to 1 to enable consistent

comparison

$$\text{Cost}_P = \sum_{i=1}^{|P|} (C_4 \times x_i + C_5 \times y_i + C_6 \times z_i)$$

$$\text{where } x_i = \begin{cases} 1, & \text{if } t_i^{\text{pick}} > tw_i^e \\ 0, & \text{otherwise} \end{cases}$$

$$y_i = \begin{cases} 1, & \text{if } \frac{t_i^{\text{drop}} - t_i^{\text{pick}}}{T(\text{spath}_i)} > \text{Dtr}'_{\max} \\ 0, & \text{otherwise} \end{cases}$$

$$z_i = \begin{cases} 1, & \text{if } \frac{|\text{path}_i|}{|\text{spath}_i|} > \text{Sta}'_{\max} \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

To enhance the interpretability of these two objectives, we applied linear normalization to the total cost according to (11). In this equation, $CB_{\min}$, $CB_{\max}$ and $CP_{\min}$, $CP_{\max}$ are the lower and upper bound of Cost_B and Cost_P , respectively. The value of these variables in each network is determined through testing. In real-world applications, they can also be obtained through estimation

$$\text{Cost} = \frac{\text{Cost}_B - CB_{\min}}{CB_{\max} - CB_{\min}} + \frac{\text{Cost}_P - CP_{\min}}{CP_{\max} - CP_{\min}}. \quad (11)$$

IV. PASSENGER ASSIGNMENT AND ROUTE DESIGN

The solving process for the proposed UCB model is presented in Algorithm 1, which consists of two consecutive phases in each iteration: scheduling phase and running phase. In the scheduling phase, we construct combined orders to serve prebooked passengers (lines 5–11 in Algorithm 1). First, we identify the unserved passenger with the earliest tw_i^s , denoted as $p_{\text{fir}} \in P$, as the initial passenger for a combined order CO_k . Second, CO_k is initialized with information of p_{fir} . Third, we utilize both the proposed ACS method and the Dijkstra-based greedy algorithm to assign additional unserved passengers to CO_k and optimize the route accordingly. Finally, the constructed CO_k is added to S_a . In the running phase (lines 12–20 in Algorithm 1), all constructed combined orders in S_a are executed by buses under uncertain traffic conditions. During execution, real-time passenger requests that satisfy the constraints may be added to buses using the greedy passenger assignment method. The Dijkstra-based greedy routing method is also used for route design. Upon completion, the solution cost Cost_a is calculated for S_a , and S_{Best} is updated if S_a yields better results. When all ant colonies complete their solution constructions, the ACS algorithm performs a global pheromone matrix update before proceeding to the next iteration (line 22 in Algorithm 1). Throughout the entire process, passenger assignment and the Dijkstra-based routing greedy method play important roles. Consequently, we delve into these two components in Sections IV-A and IV-B.

A. Passenger Assignment

Passenger assignment allocates appropriate passengers to buses. In our model, passenger assignment is performed

Algorithm 1 Process of Solving the Proposed UCB Model

Input: The number of ants A , the maximum number of iterations $iter_{max}$.

Output: The obtained best solution S_{Best} .

```

1   $iter = 0$ 
2   $Cost_{Best} = +\infty$ 
3  while  $iter < iter_{max}$  do
4      for  $a = 1, 2, \dots, A$  do
5           $S_a = \emptyset$  // Scheduling phase
6          while Not all passengers served do
7              Find the first passenger  $p_{fir}$ 
8              Initialize  $CO_k$  with the information of  $p_{fir}$ 
9              Add more unserved passengers to  $CO_k$ 
              using the ACS passenger assignment
              method and the Dijkstra-based greedy
              routing method
10              $S_a = S_a \cup \{CO_k\}$ 
11         end
12          $Cost_a = 0$  // Running phase
13         for  $CO_k \in S$  do
14             Running  $CO_k$  under uncertain traffic
              conditions while dynamically adding
              real-time passengers through a greedy
              assignment strategy and the Dijkstra-based
              greedy routing method
15              $Cost_a = Cost_a + Cost(CO_k)$ 
16         end
17         if  $Cost_a < Cost_{Best}$  then
18              $Cost_{Best} = Cost_a$ 
19              $S_{Best} = S_a$ 
20         end
21     end
22     ACS global pheromone update
23      $iter = iter + 1$ 
24 end
    
```

twice: first for prebooked passengers during the scheduling phase, then for real-time passengers during the running phase. Passenger assignment is NP-hard as it is a set partitioning problem [29]. To solve this problem effectively, we employ different methods for each case: ACS is used for prebooked passengers to achieve better global optimization performance, while a greedy method handles real-time passengers to ensure quick response to incoming requests. This section describes ACS first, followed by the greedy method.

1) *ACS Representation*: First, we introduce the devised ACS encoding scheme, which maps ant movements to solution construction. In this scheme, ant movements are classified into two types: intrastation movements and interstation movements, as depicted in Fig. 2. For intrastation movements, ants select suitable passengers within a bus station based on heuristic and pheromone information, exemplified by passengers p_7 and p_9 in Fig. 2. To avoid local optima, this selection process also considers unserved passengers from adjacent stations. For instance, passenger p_4 in Fig. 2 is selected at station A but

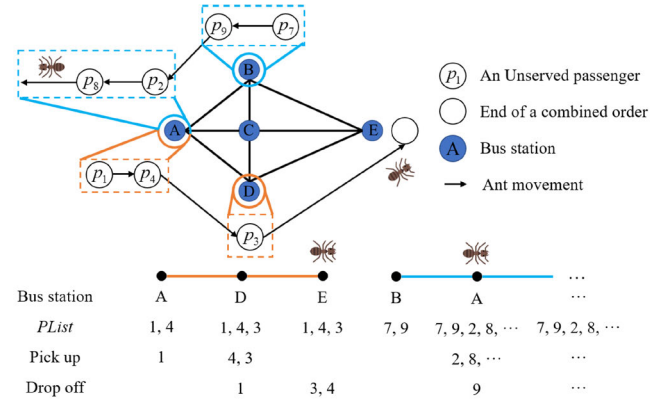


Fig. 2. Example of the movement of ants.

picked up at station D. Interstation movements correspond to bus transitions between stations, establishing connections for passengers at different locations, such as passengers p_9 and p_2 in Fig. 2. During these movements, newly added passengers are appended to the end of $PList$. After completing one combined order, the ant proceeds to construct subsequent orders until all prebooked passengers are assigned.

2) *ACS Selection*: During ant movements, correlations among passengers must be considered to minimize total cost. To this end, we construct feasible passenger sets containing passengers with spatial and temporal similarities to those already in $PList$. The maintenance of these feasible passenger sets is described below, from which ACS selects passengers. Specifically, when at station v , ACS selects the next passenger p_{next} for combined order CO_k from the feasible passenger set fea_k^v using pseudorandom proportional rules formulated in (12). Where q is a uniformly distributed random number in $[0, 1]$ and q_0 ($0 \leq q_0 \leq 1$) is a preset parameter. When $q \leq q_0$, the ant directly selects the passenger with maximum probability (exploitation); otherwise, it performs roulette selection J based on the calculated probabilities (exploration)

$$p_{next} = \begin{cases} \arg \max_{p_j \in fea_k^v} \{prob_j\}, & \text{if } q \leq q_0 \\ J, & \text{otherwise.} \end{cases} \quad (12)$$

Let p_i represent the last passenger of $PList_k$, τ denote the pheromone matrix of ACS, and η represent the heuristic information. The probability of selecting passenger p_j from the feasible passenger set fea_k^v is determined by the following equation:

$$prob_j = \frac{(\tau_{i,j}) (\eta_{i,j})^\beta}{\sum_{p_o \in fea_k^v} (\tau_{i,o}) (\eta_{i,o})^\beta}. \quad (13)$$

Let $Rou_{k,j}$ denote the modified route designed to serve passenger p_j . $Rou_{k,j}$ may remain identical to Rou_k if no route modification is required. The heuristic value $\eta_{i,j}$ is computed as the difference in estimated total travel time between Rou_k and $Rou_{k,j}$, as formalized in (14). This metric is chosen for two reasons: 1) travel time computation is computationally efficient using the average travel time of edges and 2) bus

operational costs are directly proportional to travel time, as established in (9)

$$\eta_{i,j} = \frac{1}{1 + \max(0, (T(\text{Rou}_k) - T(\text{Rou}_{k,j})))}. \quad (14)$$

After each passenger selection, ACS executes a local pheromone update according to (15), where τ_0 denotes the initial pheromone value

$$\tau_{i,\text{next}} \leftarrow (1 - \xi) \times \tau_{i,\text{next}} + \xi \times \tau_0. \quad (15)$$

3) *Greedy Selection*: For passenger assignment during the running phase, a greedy method is adopted to ensure rapid response. The greedy selection process is similar to the ACS selection process. When bus b_k arrives at station v , a feasible passenger set fea_k^v is constructed for passenger selection. Unlike ACS selection, the greedy method additionally considers the states of passengers already recorded in $PList_k$ but not yet picked up. After constructing the feasible set, the greedy method selects the next passenger p_{next} for the combined order CO_k from fea_k^v according to (16). The heuristic value $\eta_{i,j}$ is calculated according to (14), with its effectiveness being evaluated in Section V. After selection, the newly chosen passenger is added to the end of $RList_k$, and Rou_k is updated if a new route is designed to serve p_{next}

$$p_{\text{next}} = \arg \max_{p_j \in \text{fea}_k^v} \{\eta_{i,j}\}. \quad (16)$$

4) *Feasible Passenger Set Maintenance*: The maintenance process of the feasible passenger set is illustrated in Fig. 3. The process begins by considering all unserved passengers at the current station v and its unvisited adjacent stations. This approach achieves a balance between exploration and exploitation. Including passengers from adjacent stations expands the selection space, which may result in better results. But the correlation between passengers generally decreases as the distance increases. The diversity of route design and the need for timeliness make it difficult for us to consider passengers in stations that are far away from the current station.

The initial feasible passenger set comprises passengers from these candidate passengers who satisfy relevant constraints. Subsequent selections from this set are performed using either ACS (scheduling phase) or a greedy method (running phase). After each selection, the chosen passenger is removed from the set, and all remaining passengers are reevaluated against the constraints, as the combined order information updates when adding the new passenger. Passengers that still satisfy the constraints form the updated feasible passenger set for the next selection iteration. The passenger assignment process at station v terminates when either no passengers meet the constraints or the bus reaches full capacity.

The constraints for constructing the feasible passenger set primarily correspond to those already introduced in Section III-B. The first constraint is the travel time constraint. For a candidate passenger p_i , the newly designed route Rou_k^i must be less than or equal to the maximum travel time of buses, as shown in (17). Notably, the average travel time of edges is used during estimation, while the travel time from the last drop-off station to the bus depot is estimated under worst

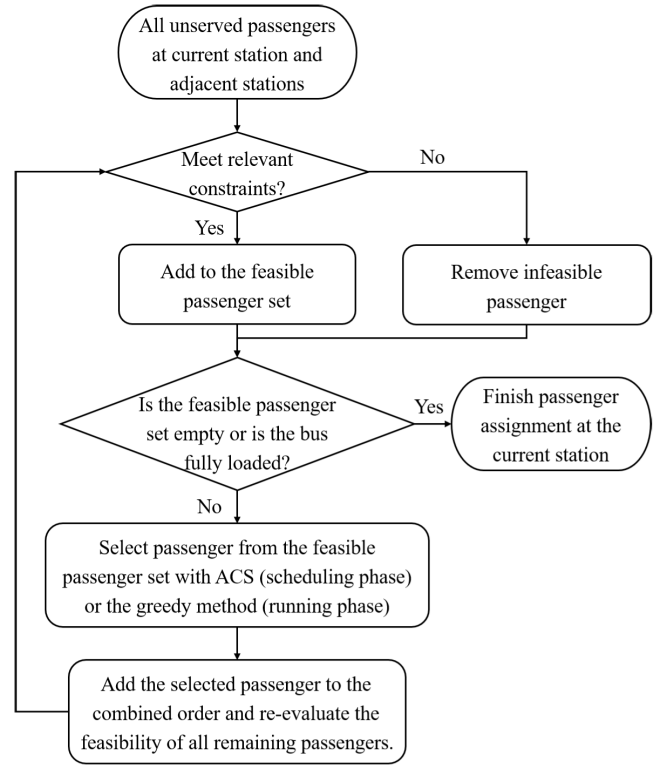


Fig. 3. Flowchart of feasible passenger set maintenance.

case traffic conditions to account for uncertainty

$$T(\text{Rou}_k^i) \leq T_{\max}. \quad (17)$$

The second constraint is the detour constraint formulated in (18), where $t_j^{\text{pick}'}$ and $t_j^{\text{drop}'}$ denote the estimated pick-up and drop-off times for passenger p_j based on $\text{Rou}_{k,j}$. Here, PSet_k^v and RSet_k^v are two sets contain passengers in $PList_k$ and $RList_k$ whose journeys are not yet complete at station v . For passengers picked up by buses, $t_j^{\text{pick}'} = t_j^{\text{pick}}$. Otherwise, $t_j^{\text{pick}'}$ is estimated using the average travel times of edges along $\text{Rou}_{k,i}$. The estimated drop-off time $t_j^{\text{drop}'}$ is also estimated using the same way

$$\frac{t_j^{\text{drop}'} - t_j^{\text{pick}'}}{T(\text{spath}_j)} \leq \text{Dtr}_{\max}$$

For $\forall p_j \in (\text{PSet}_k^v \cup \text{RSet}_k^v \cup \{p_i\})$. (18)

The third constraint is the intermediate station number constraint, formulated in (19). This constraint requires that the station ratio of relevant passengers must not exceed the upper bound $\text{Sta}_{\max}$, as specified in (6), where StaNum_j denotes the number of stations traversed when passenger p_j completes journey via route $\text{Rou}_{k,i}$

$$\frac{\text{StaNum}_j}{|\text{spath}_j|} \leq \text{Sta}_{\max}$$

For $\forall p_j \in (\text{PSet}_k^v \cup \text{RSet}_k^v \cup \{p_i\})$. (19)

In addition to the constraints in Section III-B, we also consider constraints relevant to the boarding time window of passengers. Passengers who can be picked up within their

specified boarding time windows are preferred, as formulated in (20). Where t_k^v represents the arrival time of bus b_k at station v . In the scheduling phase, this corresponds to the estimated arrival time, while in the running phase, it is the actual arrival time of b_k

$$tw_i^s \leq t_k^v + T(v, ori_i) \leq tw_i^e. \quad (20)$$

To increase the occupancy, we also pick up passengers whose boarding time window ends earlier than the arrival of bus b_k , that is, $tw_i^e < t_k^v + T(v, ori_i)$. For these passengers who will certainly incur delay costs after being picked up, we should try to avoid additional detour costs. Therefore, in this scenario, constraint (21) must be satisfied to prevent the incurrence of detour cost for passenger p_i

$$\frac{t_i^{\text{drop}'} - t_i^{\text{pick}'}}{T(\text{spath}_i)} \leq \text{Dtr}'_{\max}. \quad (21)$$

In conclusion, candidate passengers who satisfy either constraints (17), (19), (18), and (20) or constraints (17), (19), (18), and (21) can be added to fea_k^v . After each selection, the state of the combined order changes, and the remaining passengers in fea_k^v need to be reevaluated. Only those passengers who still meet constraints can be retained for the next selection. The passenger assignment process at station v terminates when either the feasible passenger set becomes empty or the bus reaches full capacity, after which the system proceeds to the next station for further passenger assignment.

5) *ACS Global Update*: When all generated solutions are evaluated by (11), ACS performs global pheromone update using the best solution found in the current iteration. Let S_{cb} be the best solution found by ACS in the current iteration, while Cost_{cb} be the corresponding cost. ACS performs a global pheromone update with these two variables according to (22). After the global update, the connection between relevant passengers will be strengthened by depositing more pheromone. Meanwhile, passengers with lower correlation will be selected with a lower probability due to evaporation

$$\begin{aligned} \tau_{i,j} &\leftarrow (1 - \rho) \times \tau_{i,j} + \rho \times \Delta\tau_{i,j} \\ \text{If the sequence } (p_i, p_j) \text{ in any } PList \text{ of } S_{cb} \\ \text{where } \Delta\tau_{i,j} &= \frac{1}{\text{Cost}_{cb}}. \end{aligned} \quad (22)$$

B. Route Design

Route design is another crucial part of UCB's operation. Route design is also an NP-hard problem since it is a constrained TSP indeed [30]. Since Route design is frequently used, we decided to solve it using a Dijkstra-based greedy method. The proposed method can efficiently solve the problem and obtain a satisfactory solution.

The pseudocode of the proposed Dijkstra-based greedy routing method is presented in Algorithm SI in the supplementary material. At first, a set staSet that contains all stations that need to be visited is constructed at first (lines 1–5 in Algorithm SI). Origins and destinations of the candidate passenger p_i and passengers in set PSet_k^v and RSet_k^v are all included. The current station v is removed from the set last

to avoid duplication. Then, we initialize the designed route with the original route (lines 6–12 in Algorithm SI). After that, we begin the construction of the route based on the Dijkstra-based greedy method (lines 14–33 in Algorithm SI). We calculate the estimated travel time between the current station and stations in staSet with the Dijkstra algorithm. Dijkstra algorithm calculates the shortest path taking into account the stations already in Rou_k^i to prevent violating assumption 1. During calculation, we will assign a travel time of infinity to a station if it is unavailable. A station is marked as unavailable if any passenger in the set $(\text{PSet}_k^v \cup \text{RSet}_k^v \cup \{p_i\})$, whose destination is that station exactly, is not picked up yet. If a bus travels to an unavailable station, some passengers cannot be transported to their destinations. That is because, according to assumption 1, buses are unable to return to this station after picking them up. At last, the station with the minimum estimated travel time is selected as the next station, and the corresponding route is added to the back-designed route. The selected station is removed from staSet to avoid duplication and set as the current station for subsequent construction. The route design process continues until either $\text{staSet} = \emptyset$ or no available station can be found. The latter case indicates that no feasible route can be designed to serve passenger p_i , meaning p_i cannot be included in the feasible passenger set.

The proposed Dijkstra-based greedy method is able to get a solution efficiently. Moreover, the method tends to produce satisfactory solutions since passengers sharing the same bus and time slot generally have similar itineraries. The efficiency and effectiveness of the proposed Dijkstra-based greedy method will be studied in Section V.

C. Overall Process

When constructing a combined order, passenger assignment and route design are two interdependent parts that mutually influence each other. The designed route determines the subsequent stations that need to be reached. As a result, only passengers at these stations or stations adjacent to these stations are able to be assigned to the combined order later. Then, to serve new passengers picked up at these stops, we may need to design a new route that is different from the previous one. An example of this interactive building process in the scheduling phase is shown in Fig. 4. During the scheduling phase, we simulate bus movements logically, even though actual physical movement only occurs during the running phase.

First, an unserved passenger with the earliest start time $p_{\text{fir}} \in P$ is selected as the first passenger of the combined order CO_k conducted by bus b_k , namely, $PList_k = [p_{\text{fir}}]$. Rou_k is then constructed by combining three route segments: 1) the shortest path from the bus depot to ori_{fir} ; 2) the passenger service route $\text{spath}_{\text{fir}}$; and 3) the shortest path from dest_{fir} back to the bus depot. For simplicity, bus depot v_{dep} in Fig. 4 is set to station A, which is also the origin of p_{fir} . The arrival time of bus is set to $t_k^A = tw_{\text{fir}}^s$. Second, a feasible passenger set fea_k^A is constructed based on the proposed Dijkstra-based greedy routing method. We employ the proposed Dijkstra-based greedy routing method to design routes for nearby unserved passengers. With the designed routes, we can check whether these

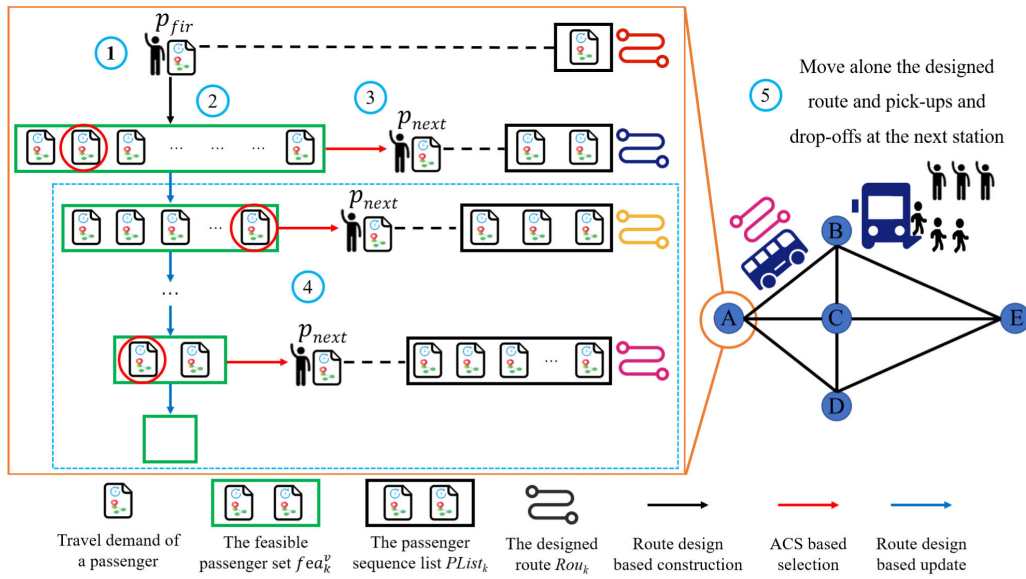


Fig. 4. Example of building a combined order in the scheduling phase. First, p_{fir} is selected as the first passenger of a combined order. Second, the feasible passenger set is constructed. Third, passenger assignment is conducted by ACS. Fourth, the process of route design-based update and ACS-based selection alternates until the feasible set is empty or the bus is full. Fifth, buses move along the designed route and provide service at every station.

passengers can be added to fea_k^A according to constraints listed in Section IV-A.4. Third, we select a passenger from fea_k^A and update relevant variables. ACS selects the most appropriate next passenger from fea_k^A with the guidance of pheromone and heuristic information. Then, the selected next passenger p_{next} is added to the back of $PList_k$. Followed by updating Rou_k if a new path is designed for serving p_{next} . For time variables, if $ori_{next} = A$, we set $t_k^A = \min(\max(t_k^A, tw_{next}^s), tw_{fir}^e)$. If ori_{next} is the adjacent station of A , we need to consider the estimated travel time in the formula. Namely, $t_k^{v_0} = \min(\max(t_k^{v_0}, tw_{next}^s - T(v_0, ori_{next})), tw_{fir}^e)$. Fourth, the process of performing ACS-based selection and updating the feasible passenger set alternates. After selection, some passengers in the feasible passenger set may no longer satisfy relevant constraints. Therefore, we update fea_k^A to perform the next selection. Selection and update processes alternate until the $fea_k^A = \emptyset$ or bus b_k is full. Fifth, b_k continues moving to the next stations according to the designed route and provides pick-up and drop-off service at these stations. Once the passenger assignment at station A is finished, b_k proceeds to station B following Rou_k along the road network edges. Upon reaching station B , passengers whose destination is station B are dropped off. Bus b_k also constructs a feasible passenger set fea_k^B at station B for passenger assignment if it is not fully loaded. Notably, fea_k^B is constructed based on information regarding assigned passengers who have not alighted at station B . Following the pick-ups and drop-offs at station B , b_k continues to the subsequent stations, updating relevant variables until all passengers have been dropped off. During the moving process, the stopping time and restarting time of the bus (t_s and t_r), and the boarding time and alighting time of the passenger (t_b , t_a) are all considered.

In the running phase, the construction of $RList_k$ for the combined order CO_k follows a similar process as described previously. If the first station in the route is not the bus depot,

the bus must depart from the depot with a lead time t_{ahead} (3 mins) to account for potential traffic delays. Upon arriving at each station, the feasible passenger set is constructed considering all unfinished passengers from both $PList_k$ and $RList_k$. The greedy passenger assignment method is then performed to select passengers for $RList_k$, followed by updates to CO_k and the feasible passenger set, a process identical to that in the scheduling phase.

V. EXPERIMENT

In this section, we conduct a series of experiments to demonstrate the superiority of the proposed method. First, we provide a brief introduction to the experimental setup. Then, we prove the effectiveness of the proposed heuristic information. Next, we compare the proposed algorithm with several baseline algorithms. Following this, we evaluate both the efficiency and performance of our Dijkstra-based greedy routing approach. Finally, we examine the robustness of our method under varying levels of uncertainty associated with the urban traffic condition.

A. Experimental Settings

We conduct experiments on three types of road networks: realistic-based networks from the transportation networks for research (TNTP) [31], computer-generated scale-free networks [32], and a self-constructed road network. From the TNTP datasets, we select four networks: Sioux Falls (SF, 24 nodes, 38 edges), a subgraph of Anaheim (AN, 42 nodes, 55 edges), a subgraph of Gold Coast (GC, 60 nodes, 66 edges), and Eastern Massachusetts (EMA, 74 nodes, 129 edges). For scale-free networks, we use two generated instances: SF35 (35 nodes, 99 edges) and SF50 (50 nodes, 144 edges). Our self-constructed network is constructed from the higher education mega center in Guangzhou, China, comprising 67 bus stops and 111 edges, which we named as DXC. The passenger

TABLE I
 STUDY OF HEURISTIC INFORMATION ON SF50

Situation	Best result	Difference	$\lambda = 0.1$	$\lambda = 0.5$	$\lambda = 1$	$\lambda = 10$	$\lambda = 20$	$\lambda = 50$	Mean (max - min)
$\beta = 0$	0.640543	+36.85%	/	/	/	/	/	/	0.640543 (0.0)
$\eta_{i,j} = \frac{\lambda}{StaNum_j}$	0.598746	+27.92%	0.629147	0.616466	0.619424	0.636048	0.598746	0.634435	0.622378 (0.0373)
$\eta_{i,j} = \frac{\lambda}{1+max(0, (T(Rou_k) - T(Rou_{k,j})))}$	0.468064	0.0%	0.481071	0.482364	0.495058	0.515541	0.468064	0.474332	0.486072 (0.0475)
$\eta_{i,j} = \frac{1}{StaNum_j} + \frac{\lambda}{1+max(0, (T(Rou_k) - T(Rou_{k,j})))}$	0.468897	+0.18%	0.593431	0.573959	0.5378	0.468897	0.509712	0.470487	0.525714 (0.1245)

 TABLE II
 COMPARISON RESULTS WITH REGARD TO OTHER COMPARISON ALGORITHMS ON SEVEN INSTANCES

	SF	AN	GC	EMA	SF35	SF50	DXC
ACS	0.454467	0.607576	0.498113	0.404484	0.518459	0.495058	0.450781
Greedy	0.964776 (+112.29%)	1.068185 (+75.81%)	0.966759 (+94.08%)	0.856837 (+111.83%)	0.986198 (+90.22%)	1.023742 (+106.79%)	0.864906 (+91.87%)
AS	0.661798 (+45.62%)	0.792325 (+30.41%)	0.689292 (+38.38%)	0.695051 (+71.84%)	0.702909 (+35.58%)	0.758982 (+53.31%)	0.580246 (+28.72%)
EAS	0.549671 (+20.95%)	0.660747 (+8.75%)	0.606966 (+21.85%)	0.591041 (+46.12%)	0.612818 (+18.2%)	0.638677 (+29.01%)	0.544073 (+20.7%)
ASRank	0.493876 (+8.67%)	0.684803 (+12.71%)	0.54126 (+8.66%)	0.535223 (+32.32%)	0.542623 (+4.66%)	0.581562 (+17.47%)	0.511806 (+13.54%)
AACO	0.654935 (+44.11%)	0.758182 (+24.79%)	0.704051 (+41.34%)	0.744654 (+84.1%)	0.706446 (+36.26%)	0.719098 (+45.26%)	0.610652 (+35.47%)
MAACO	0.674828 (+48.49%)	0.784842 (+29.18%)	0.721753 (+44.9%)	0.680281 (+68.18%)	0.685613 (+32.24%)	0.718424 (+45.12%)	0.649266 (+44.03%)
SPSO	0.738391 (+62.47%)	0.829867 (+36.59%)	0.820076 (+64.64%)	0.86435 (+113.69%)	0.763204 (+47.21%)	0.785567 (+58.68%)	0.666805 (+47.92%)
SLLSO	0.726346 (+59.82%)	0.830122 (+36.63%)	0.795794 (+59.76%)	0.848506 (+109.77%)	0.723096 (+39.47%)	0.763816 (+54.29%)	0.670892 (+48.83%)

data for SF, AN, GC, EMA, SF35, and SF50 networks are synthetically generated based on predefined rules. For DXC, the prebooked passenger data originated from smart card records provided by the bus operator of the higher education mega center, comprising 1,025 actual trip records. For real-time passenger data of DXC, it is also generated based on predefined rules. The diagrams of these networks and other settings are provided in the supplementary material. All experimental results presented in this article represent averages obtained from 10 independent runs.

B. Effectiveness of Heuristic Information

Heuristic information is an important part of ACO. In this study, heuristic information is also adopted by the greedy passenger assignment method for passenger selection. Heuristic information is usually a kind of problem-specific information that is used to guide an algorithm to converge in a faster and better way. In this section, we experiment on SF and SF50 to prove the effectiveness of the proposed heuristic information calculated according to (14). We consider four different situations in this experiment. In situation 1, we set $\beta = 0$, indicating that ants do not consider heuristic information during selection. In situation 2, we use the number of reached stations for passenger p_i along $Rou_{k,i}$ as heuristic information. Because it has an impact on the cost of passengers according to (10). Situation 3 is exactly (14) introduced before. For situation 4, we take both situation 2 and situation 3 into account. A parameter λ is introduced to balance the impact between these two parts. Results on SF50 are presented in Table I, including results of different λ values in situation 2 and situation 3.

From Table I, we can see that the heuristic information can lead ACS to better results. The best results of situation 3 and situation 4 are similar, which is better than situation 2. The results of situation 3 are not so sensitive to the setting of λ compared with situation 4. What is more, the results of

situation 4 are better when λ is large, indicating the importance of (14). So, we select the travel time of $Rou_{k,j}$ to calculate heuristic information. The results on SF are presented in Table SI in the supplementary material. For SF, the results across four different situations are similar because SF has a smaller scale, making it much easier to optimize.

C. Comparison Experiment

In this section, we compare the proposed method with different baseline algorithms. The parameters of these baseline algorithms are mainly configured according to the original authors' settings. Detailed descriptions of the parameter settings for these algorithms are available in the supplementary material. These algorithms can be categorized into three groups. The first group is the greedy method, which always selects passengers with maximum heuristic information value at each step. The second group is the ACO methods, including the ant system (AS) [25], the elite AS (EAS) [33], the ranked AS (ASRank) [34], the adaptive ant colony optimization (AACO) [35], and the modified adaptive ant colony optimization algorithm (MAACO) [36]. The third group is the set-based particle swarm optimization (SPSO) [37] and set-based level-based learning swarm optimizer (SLLSO) [38]. For fair comparison, we ensure identical evaluation numbers across all algorithms in the second and third groups.

The experimental results are presented in Table II. We highlight results that show statistically significant differences according to the Wilcoxon rank-sum test at a 0.05 significance level. From Table II, we can see that the greedy method, which only considers the heuristic information, tends to get trapped in local optima since it does not fully consider the correlation among different passengers. As a result, it performs the worst among all methods in all cases. The six different ACO methods, on the other hand, guide ants using both heuristic and pheromone information. The pheromone matrix stores information about some of the best solutions obtained

TABLE III
COMPARATIVE RESULTS OF ROUTING ALGORITHMS

	time (s)	bestWin	bestEq	bestLost	avgWin	avgEq	avgLost
DG-R	0.01						
500-ACS-R	0.827	3	19	19	3	3	35
1000-ACS-R	1.625	3	22	16	3	6	32
2000-ACS-R	3.228	5	25	11	4	8	29
5000-ACS-R	7.921	5	29	7	4	10	27
10000-ACS-R	15.763	5	29	7	5	12	24
DG-R	0.02						
500-ACS-R	2.693	2	27	52	1	5	75
1000-ACS-R	5.217	3	33	45	2	11	68
2000-ACS-R	10.13	3	38	40	3	18	60
5000-ACS-R	24.762	4	46	31	4	27	50
10000-ACS-R	47.118	5	54	22	5	33	43

through the global update process of ACO. Passengers in combined orders with lower cost have a higher probability of being selected in the next generation due to the influence of the pheromone matrix. Thus, the correlation among passengers is indirectly represented by the pheromone matrix. The differences in results among the ACO methods are mainly caused by variations in selection and pheromone updating mechanisms. ACS, with its pseudorandom selection, local update, and global update processes, exhibits better diversity and optimization capabilities, leading to superior results. Another crucial mechanism is elitism. In ACS, EAS, and ASRank, elitism is employed to enable only some of the best ants to deposit pheromones, resulting in improved outcomes. The absence of elitism causes AS, AACO, and MAACO to perform worse in large-scale search scenarios constructed in this article. As for SPSO and SLLSO, the obtained results are generally better than the greedy method but worse than the ACO methods. This is because it is challenging for the offspring of SPSO and SLLSO to effectively learn from historical information. In SPSO and SLLSO, the position of a particle represents a set of elements that constitute a feasible solution to the problem. The subtraction operation between two positions is defined as the relative complement of the two sets. In the case of the constrained passenger assignment problem, feasible solutions tend to converge to similar ones after several iterations, resulting in only a few elements remaining after the subtraction operation. Consequently, the offspring struggles to learn from historical information and can only select passengers from the entire feasible set in a greedy manner.

D. Routing Algorithms Comparison

In this section, we evaluate the performance of the proposed Dijkstra-based greedy routing method (DG-R) against the ACS routing method (denoted as ACS-R to distinguish it from the ACS passenger assignment method). The comparison is conducted using 41 routing tasks from SF and 81 tasks from DXC. Taking into account the inherent randomness in ACS-R, we conduct 10 independent experimental runs for ACS-R with varying evaluation numbers, recording the optimal and average results. The results are presented in Table III, revealing two key findings. First, DG-R demonstrates significantly faster computation times than ACS-R, owing to its greedy mechanism. Second, DG-R achieves solution quality comparable

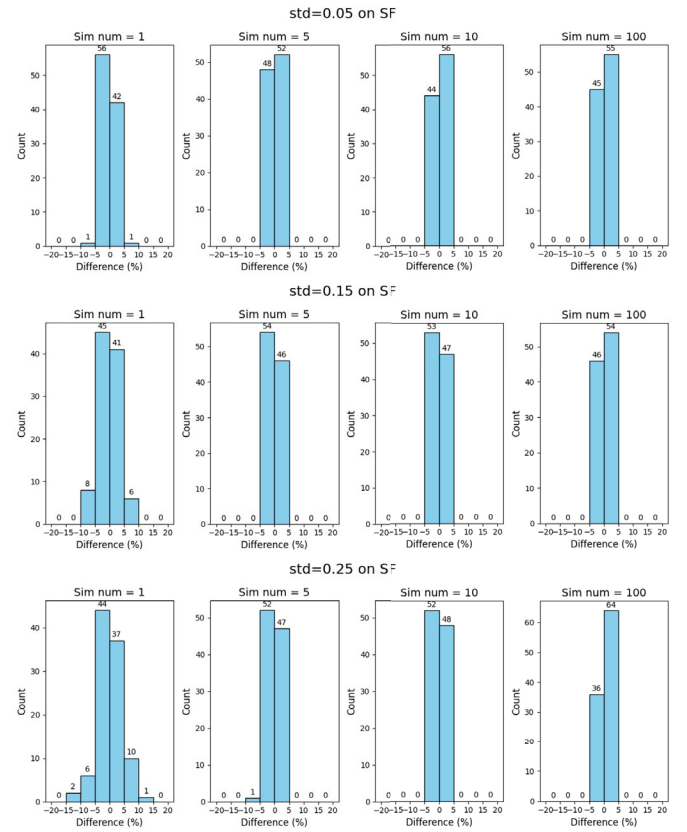


Fig. 5. Results of uncertainty study on SF.

to ACS-R, performing worse in only a few cases. This is because passengers assigned to the same bus typically have similar itineraries, and the shortest paths identified by the DG-R generally yield satisfactory results.

E. Uncertainty Study

In this section, we analyze the influence of different uncertainty levels on urban traffic conditions. We test standard deviation values of $\text{std} = 0.05, 0.15, 0.25$ for the truncated normal distribution in both SF and DXC. To evaluate the impact of uncertainty, we randomly generate 100 solutions and assess them through Monte Carlo simulation with varying simulation counts ($\text{sim} = 1, 5, 10, 100, 1000$), recording the average cost for each configuration. During each simulation for every solution, we sample the actual travel time for each edge at minute-level intervals. The results from $\text{sim} = 1000$ serve as the ground truth, and we calculate percentage differences between these reference values and those obtained with $\text{sim} = 1, 5, 10, 100$. For each std value, we categorize solutions according to their difference intervals. The SF results are presented in Fig. 5, while DXC results appear in Fig. S8 of the supplementary material. Fig. 5 reveals that higher std values correspond to greater uncertainty impacts. However, in most cases, the results from $\text{sim} = 1, 5, 10, 100$ remain similar to the $\text{sim} = 1000$ baseline across all cases. The differences predominantly fall within the $[-5\%, 5\%]$ range, even for $\text{sim} = 1$. This phenomenon occurs because the random variations in edge travel times tend to cancel each

other out, causing the results to regress toward the mean value. The conclusions drawn from DXC are the same as those of SF. Therefore, sim is set to 1 in other experiments.

VI. CONCLUSION

In this article, we build a UCB model with flexible pick-ups and drop-offs. In our model, when a bus passes through a station, it can drop off passengers whose destination is exactly this station and pick up new passengers as long as constraints are satisfied. The passenger assignment and route design of buses are optimized using the devised ACS and Dijkstra-based greedy methods, respectively, considering both operational cost and passenger experience. The proposed flexible model enables the utilization of UCB in various scenarios, such as peak commuting hours, off-peak periods, holidays, and across a wider range of locations, without the need to differentiate between boarding and alighting areas. For the limitation of this work, we consider fuel-powered buses as the vehicles providing service in the current study. However, with the rapid development of electric vehicles driven by energy conservation and environmental protection demands, electric buses are becoming increasingly prevalent in urban areas. Providing UCB services with electric buses will likely become a prominent trend in the near future. When using electric buses to provide UCB service, time-of-use electricity pricing and vehicle-to-grid technology necessitate the design of effective algorithms to reduce operational cost, presenting significant opportunities for further research.

REFERENCES

- [1] T. Liu and A. Ceder, "Analysis of a new public-transport-service concept: Customized bus in China," *Transp. Policy*, vol. 39, pp. 63–76, Apr. 2015.
- [2] P. Vansteenwegen, L. Melis, D. Aktaş, B. D. G. Montenegro, F. Sartori Vieira, and K. Sörensen, "A survey on demand-responsive public bus systems," *Transp. Res. C, Emerg. Technol.*, vol. 137, Apr. 2022, Art. no. 103573.
- [3] D. Huang, Y. Gu, S. Wang, Z. Liu, and W. Zhang, "A two-phase optimization model for the demand-responsive customized bus network design," *Transp. Res. C, Emerg. Technol.*, vol. 111, pp. 1–21, Feb. 2020.
- [4] W. Shu and Y. Li, "A novel demand-responsive customized bus based on improved ant colony optimization and clustering algorithms," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 8, pp. 8492–8506, Aug. 2023.
- [5] R. Guo, W. Zhang, W. Guan, and B. Ran, "Time-dependent urban customized bus routing with path flexibility," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2381–2390, Apr. 2021.
- [6] Y. Zhao, Z. Feng, and X. Li, "Transfer-based customized bus network design with holding control and heterogeneous fleet," *IEEE Trans. Evol. Comput.*, vol. 29, no. 6, pp. 2567–2581, Dec. 2025.
- [7] D. Ma, B. Fang, W. Ma, X. Wu, and S. Jin, "Potential routes extraction for urban customized bus based on vehicle trajectory clustering," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 11878–11888, Nov. 2023.
- [8] B. Wu, X. Zuo, G. Chen, G. Ai, and X. Wan, "Multi-agent deep reinforcement learning based real-time planning approach for responsive customized bus routes," *Comput. Ind. Eng.*, vol. 188, Feb. 2024, Art. no. 109840.
- [9] R. Guo, W. Guan, M. Vallati, and W. Zhang, "Modular autonomous electric vehicle scheduling for customized on-demand bus services," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 10055–10066, Sep. 2023.
- [10] R. Guo, W. Guan, A. Huang, and W. Zhang, "Exploring potential travel demand of customized bus using smartcard data," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Oct. 2019, pp. 2645–2650.
- [11] D. Xia, L. Zheng, X. Cai, W. Liu, and D. Sun, "Urban customized bus design for private car commuters," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21723–21735, Nov. 2022.
- [12] X. Chen, Y. Wang, and X. Ma, "Integrated optimization for commuting customized bus stop planning, routing design, and timetable development with passenger spatial-temporal accessibility," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2060–2075, Apr. 2021.
- [13] C. Ma, C. Wang, and X. Xu, "A multi-objective robust optimization model for customized bus routes," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2359–2370, Apr. 2021.
- [14] X. Chen, Y. Wang, Y. Wang, X. Qu, and X. Ma, "Customized bus route design with pickup and delivery and time windows: Model, case study and comparative analysis," *Expert Syst. Appl.*, vol. 168, Apr. 2021, Art. no. 114242.
- [15] W. Li, L. Zheng, L. Liao, X. Yang, D. Sun, and W. Liu, "A multiline customized bus planning method based on reinforcement learning and spatiotemporal clustering algorithm," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 3, pp. 3691–3705, Jun. 2024.
- [16] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, Apr. 1997.
- [17] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Math.*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [18] Q. Liu, Q. Li, C. Tang, H. Lin, X. Ma, and T. Chen, "A visual analytics approach to scheduling customized shuttle buses via perceiving passengers' travel demands," in *Proc. IEEE Visualizat. Conf. (VIS)*, Oct. 2020, pp. 76–80.
- [19] L. Melis and K. Sörensen, "The static on-demand bus routing problem: Large neighborhood search for a dial-a-ride problem with bus station assignment," *Int. Trans. Oper. Res.*, vol. 29, no. 3, pp. 1417–1453, May 2022.
- [20] H. Liu, J. Zhang, and M. Zhou, "Adaptive particle swarm optimizer combining hierarchical learning with variable population," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 3, pp. 1397–1407, Mar. 2023.
- [21] C. Han, Y. Gu, G. Wu, and X. Wang, "Simulated annealing-based heuristic for multiple agile satellites scheduling under cloud coverage uncertainty," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 5, pp. 2863–2874, May 2023.
- [22] X. Wang, T.-M. Choi, H. Liu, and X. Yue, "A novel hybrid ant colony optimization algorithm for emergency transportation problems during post-disaster scenarios," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 48, no. 4, pp. 545–556, Apr. 2018.
- [23] Y.-H. Jia et al., "An intelligent cloud workflow scheduling system with time estimation and adaptive ant colony optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 1, pp. 634–649, Jan. 2021.
- [24] M. Asghari, S. M. J. M. Al-E-Hashem, and Y. Rekik, "Environmental and social implications of incorporating carpooling service on a customized bus system," *Comput. Oper. Res.*, vol. 142, Jun. 2022, Art. no. 105724.
- [25] A. Colomni, M. Dorigo, and V. Maniezzo, "Distributed optimization by ant colonies," in *Proc. 1st Eur. Conf. Artif. Life*, vol. 142, Dec. 1991, pp. 134–142.
- [26] M. A. Nayeem, M. M. Islam, and X. Yao, "Solving transit network design problem using many-objective evolutionary approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 10, pp. 3952–3963, Oct. 2019.
- [27] J. M. López-Martínez et al., "Modelling the fuel consumption and pollutant emissions of the urban bus fleet of the city of Madrid," *Transp. Res. D, Transp. Environ.*, vol. 52, pp. 112–127, May 2017.
- [28] B. Y. Chen, C. Shi, J. Zhang, W. H. K. Lam, Q. Li, and S. Xiang, "Most reliable path-finding algorithm for maximizing on-time arrival probability," *Transportmetrica B, Transp. Dyn.*, vol. 5, no. 3, pp. 248–264, Jul. 2017.
- [29] M. Lewis, G. Kochenberger, and B. Alidaee, "A new modeling and solution approach for the set-partitioning problem," *Comput. Oper. Res.*, vol. 35, no. 3, pp. 807–813, Mar. 2008.
- [30] X. Meng, J. Li, X. Dai, and J. Dou, "Variable neighborhood search for a colored traveling salesman problem," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 4, pp. 1018–1026, Apr. 2018.
- [31] *Transportation Networks for Research*. Accessed: Jun. 20, 2025. [Online]. Available: <https://github.com/bstabler/TransportationNetworks>
- [32] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999.
- [33] M. Dorigo, "Optimization, learning and natural algorithms," Ph.D. thesis, Dipartimento di Elettronica, Politecnico di Milano, Milan, Italy, 1992.
- [34] B. Bullnheimer, R. F. Hartl, and C. Strauß, "A new rank based version of the ant system. A computational study," *Central Eur. J. Oper. Res. Econ.*, vol. 7, no. 1, pp. 25–38, 1997.

- [35] P. Stodola and J. Nohel, "Adaptive ant colony optimization with node clustering for the multidepot vehicle routing problem," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1866–1880, Dec. 2023.
- [36] L. Wu, X. Huang, J. Cui, C. Liu, and W. Xiao, "Modified adaptive ant colony optimization algorithm and its application for solving path planning of mobile robot," *Expert Syst. Appl.*, vol. 215, Apr. 2023, Art. no. 119410.
- [37] W.-J. Qiu, X.-M. Hu, A. Song, J. Zhang, and W.-N. Chen, "A scalable parallel coevolutionary algorithm with overlapping cooperation for large-scale network-based combinatorial optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 8, pp. 4806–4818, Aug. 2024.
- [38] Q. Yang, W.-N. Chen, J. D. Deng, Y. Li, T. Gu, and J. Zhang, "A level-based learning swarm optimizer for large-scale optimization," *IEEE Trans. Evol. Comput.*, vol. 22, no. 4, pp. 578–594, Aug. 2018.



Wen-Jin Qiu (Student Member, IEEE) received the bachelor's degree in computer science and technology from South China University of Technology, Guangzhou, China, in 2021, where he is currently pursuing the Ph.D. degree.

His current research interests include the carpool service problem, customized bus routing, and cooperative coevolution.



Zhan-Xian Liang (Student Member, IEEE) received the B.E. degree in computer science and technology from South China University of Technology, Guangzhou, China, in 2023.

Since 2023, he has served as a Client Development Engineer at ByteDance Company Ltd., Beijing, China.



Xiao-Min Hu (Senior Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Sun Yat-sen University, Guangzhou, China, in 2006 and 2011, respectively.

She is currently a Professor with the School of Computer Science and Technology, Guangdong University of Technology, Guangzhou. She has published over 60 papers in international journals and conferences. Her current research interests include artificial intelligence, data mining, evolutionary computation, swarm intelligence, and their applications in design and optimization on wireless sensor networks, route optimization, and big data analysis.

Dr. Hu was a recipient of the Pearl River S&T Nova Program of Guangzhou in 2018. She serves as an Associate Editor for IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE.



Jun Zhang (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the City University of Hong Kong, Hong Kong, in 2002.

He has authored more than 600 peer-reviewed publications, including over 230 papers in IEEE TRANSACTIONS. He has been recognized as a Clarivate Highly Cited Researcher, ranking within the top 1% worldwide in the field of computer science in 2024. His primary research interests lie in the field of computational intelligence (CI), with a strong focus on evolutionary computation (EC) and its wide-ranging applications.

Dr. Zhang was awarded the Outstanding Young Scientist Fund by the National Natural Science Foundation of China (NSFC), in 2011. He currently serves as an Associate Editor of IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE, IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, and IEEE TRANSACTIONS ON CYBERNETICS.



Wei-Neng Chen (Senior Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Sun Yat-sen University, Guangzhou, China, in 2006 and 2012, respectively.

He is currently a Full Professor with the School of Computer Science and Engineering, South China University of Technology, Guangzhou. He has co-authored over 200 international journal and conference papers, including more than 100 papers published in IEEE TRANSACTIONS journals. His current research interests include computational intelligence, swarm intelligence, network science, and their applications.

Prof. Chen is currently the Vice-Chair of the IEEE Guangzhou Section and the Chair of the IEEE SMC Society Guangzhou Chapter. He is also a Committee Member of the IEEE CIS Emerging Topics Task Force. He serves as an Associate Editor for IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE and IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION.